

Location-Based Database System

A location-based database system (also known as a **spatial** or **geospatial database**) is a specialized database optimized to store, query, and manipulate geographic coordinates and spatial shapes.

Common Geo Query Types:

1. **Distance query** (Find shops within 2 km)
2. **Nearest-neighbor query** (Find the nearest driver)
3. **Polygon query** (Users inside a delivery zone / Users inside a city boundary)
4. **Nearby search** (Show all drivers near me)
5. **Route / path query** (Shortest road path from A to B)

Traditional database indexes like **B-Tree** are inefficient for geographical queries — they're built for exact matches and ordered ranges on a single value, not "everything within a curved region of 2D space."

Geospatial systems instead use specialized spatial indexes such as:

- R-Tree
- QuadTree
- KD-Tree
- GeoHash
- S2 Geometry
- H3 Hexagonal Indexing

Phase 1: Distance Calculation

Step 1: Euclidean Distance

Euclidean distance is the straight-line length between two points in a plane or multi-dimensional space

2D Space: $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

3D Space: $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$

n-Dimensional Space: $d = \sqrt{(\sum \text{from } i=1 \text{ to } n \text{ of } (x_{2,i} - x_{1,i})^2)}$

Why square the difference and then take the square root?

1. **Makes negative differences positive.** Distance shouldn't be negative.
2. **Combines perpendicular directions correctly.** A 3-unit horizontal movement and a 4-unit vertical movement isn't a distance of $3 + 4 = 7$ — they're perpendicular components, so the actual straight-line distance is $\sqrt{3^2 + 4^2} = 5$.
3. **The square root undoes the squaring.** After squaring, the result is in squared units: $3^2 + 4^2 = 25$, which is effectively in units². Taking the square root brings it back to the original unit: $\sqrt{25} = 5$

Worked example — two points A = (1, 2) and B = (4, 6):

$$\Delta x = 4 - 1 = 3$$

$$\Delta y = 6 - 2 = 4$$

B (4,6)

*

| \

4 | \ 5 ← distance

| \

| \

(4,2) 3

Pythagorean theorem:

$$a^2 + b^2 = c^2$$

$$\Rightarrow 3^2 + 4^2 = d^2$$

$$\Rightarrow 25 = d^2$$

$$\Rightarrow d = \sqrt{25} = 5$$

$$\text{Therefore: } d = \sqrt{(\Delta x)^2 + (\Delta y)^2}$$

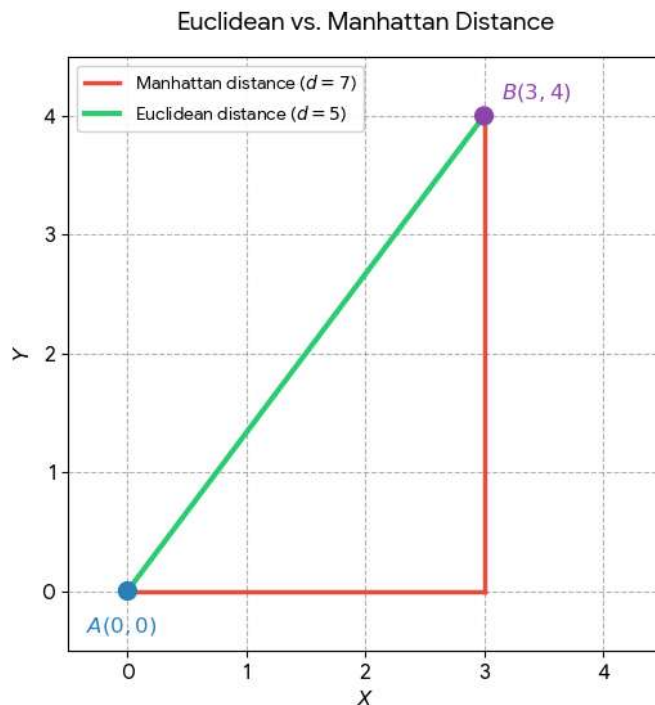
Applied to latitude/longitude as a flat-plane approximation:

```
public double distanceKm(lat1, lng1, lat2, lng2) {  
    double avgLatRad = Math.toRadians((a1 + a2) / 2.0);  
    double dLatKm = (a2 - a1) * KM_PER_DEGREE;  
    double dLngKm = (o2 - o1) * KM_PER_DEGREE  
        * Math.cos(avgLatRad);  
    double distance = Math.sqrt(  
        dLatKm * dLatKm + dLngKm * dLngKm  
    );  
    return distance;  
}
```

Step 2: Euclidean vs Manhattan Distance

Euclidean distance calculates the straight-line ("as the crow flies") distance; Manhattan distance calculates the grid-like ("city block") distance.

Feature	Euclidean Distance	Manhattan Distance
Analogy	Flying directly over a landscape.	Navigating grid-based city blocks.
Formula (2D)	$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$	$d = x_2 - x_1 + y_2 - y_1 $
Shortest path?	Always the absolute shortest path.	Shortest path restricted to 90° turns.
Outlier sensitivity	Highly sensitive (differences are squared).	Less sensitive (differences are linear).
Dimensionality	Suffers from the "curse of dimensionality" in very high dimensions.	Often works well with high-dimensional data (e.g. text classification).
Typical use	Physical geography, continuous variables where diagonal movement is natural — e.g. K-Means Clustering.	Grid-restricted movement (chess pieces, city navigation) or high-dimensional space; handles outliers and noise better since differences aren't squared — e.g. KNN.



Step 3: Haversine Distance

The Haversine distance exists because both Euclidean and Manhattan distance assume a flat, two-dimensional plane, whereas the Earth is a curved sphere.

If you used Euclidean distance to calculate the path between London and New York, your straight line would technically cut through the Earth's crust rather than follow its surface. The Haversine formula solves this by accounting for the Earth's curvature to find the true **great-circle distance** across the surface:

$$d = 2R \cdot \arctan(\sqrt{a / (1 - a)})$$

You cannot simply do: $d = \sqrt{((lat_2 - lat_1)^2 + (lon_2 - lon_1)^2)}$

because latitude and longitude are angles, not distances. For example:

- 1° of latitude $\approx$ 111 km
- 1° of longitude $\approx$ 111 km at the equator
- 1° of longitude becomes progressively smaller as you move toward the poles

```
public class HaversineDistanceCalculator implements DistanceCalculator {
    private static final double EARTH_RADIUS_KM = 6371.0;

    public double distance(double lat1, double lon1, double lat2, double lon2) {
        double dLat = Math.toRadians(lat2 - lat1);
        double dLon = Math.toRadians(lon2 - lon1);

        double a = Math.sin(dLat / 2) * Math.sin(dLat / 2)
            + Math.cos(Math.toRadians(lat1))
            * Math.cos(Math.toRadians(lat2))
            * Math.sin(dLon / 2)
            * Math.sin(dLon / 2);
        double c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
        return EARTH_RADIUS_KM * c;
    }
}
```

Phase 2: Radius Search

Step 4: The Naive Approach

If you have 10 million restaurants, you don't want to calculate Haversine distance against all 10 million of them.

Store database

```
-----  
Store 1 → (17.3850, 78.4867)  
Store 2 → (17.4100, 78.5000)  
Store 3 → (17.3600, 78.5100)  
...  
Store 10,000,000
```

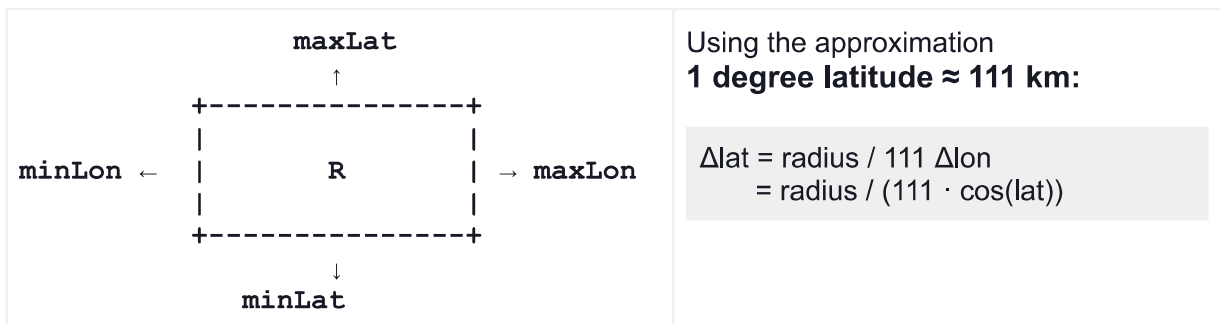
A naive query is:

```
For every store:  
  calculate Haversine(driver, store)  
  if distance <= 5 km:  
    return driver
```

Complexity: $O(N)$

Step 5: Bounding Box Optimization

Before running exact distance calculations, reduce the search area to a rectangle first:



Using the approximation
1 degree latitude $\approx$ 111 km:

$$\begin{aligned}\Delta\text{lat} &= \text{radius} / 111 \\ \Delta\text{lon} &= \text{radius} / (111 \cdot \cos(\text{lat}))\end{aligned}$$

So for a 5 km radius: $\Delta\text{latitude} \approx 5/111 \approx 0.045^\circ$. Longitude needs the extra $\cos(\text{lat})$ correction because a degree of longitude covers less ground the further you are from the equator.

Then query only the rectangle:

```
WHERE latitude BETWEEN minLat AND maxLat  
AND longitude BETWEEN minLon AND maxLon
```

This is much cheaper than calculating Haversine for every rec

Longitude is different from latitude.

At the equator: 1 degree longitude $\approx$ 111 km. But as we move toward the poles, the physical distance represented by one degree of longitude becomes smaller. The correction factor is $\cos(\text{latitude})$: **km per degree longitude $\approx 111 \times \cos(\text{latitude})$**

Therefore:

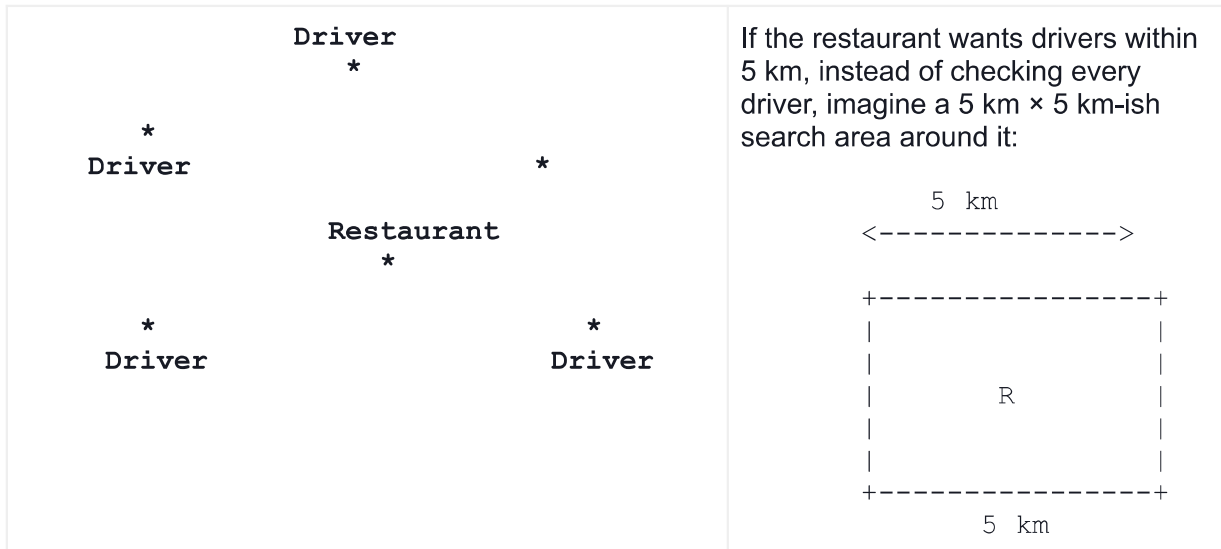
$$\text{deltaLon} = \text{radiusKm} / (111 \times \cos(\text{latitude}))$$

```
public class GeoUtils {  
    public double[] getBox(lat, lon, radiusKm) {  
        double deltaLat = radiusKm / 111.0;  
        double deltaLon = radiusKm / (111.0 * Math.cos(Math.toRadians(lat)));  
        return new double[]{  
            lat - deltaLat, // minLat  
            lat + deltaLat, // maxLat  
            lon - deltaLon, // minLon  
            lon + deltaLon // maxLon  
        };  
    }  
}
```

Phase 3: Spatial Indexing with a Grid

Step 6: Why Spatial Indexing

Stop scanning everything — this is where real geo databases begin. Imagine a normal flat map, ignoring Earth's curvature for a moment:



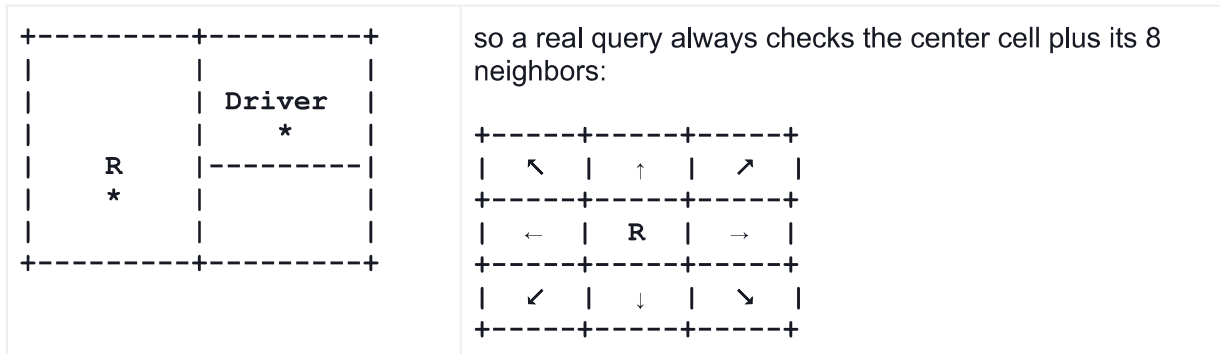
Now you only need to consider drivers inside that region — but you still need a fast way to find those drivers. That's what a grid index gives you.

Dividing the world into cells:



If a restaurant is in Cell 892, you don't search the entire database — you search Cell 892 + neighboring cells. (This exact idea, taken to a hierarchical, string-encoded extreme, is what GeoHash does — see Phase 4.)

Never search only the exact cell. A restaurant and driver can be physically close but fall in different cells right at a boundary:



Putting it together — a restaurant requests nearby drivers within 5 km:

```

Restaurant coordinates
    ↓
Spatial index
    ↓
Find candidate locations      (e.g. maybe 500 drivers)
    ↓
Exact distance calculation
    ↓
Drivers actually within 5 km  (e.g. 120 of them)
  
```

10,000,000 drivers becomes roughly 500 candidates via the spatial index, then ~120 after an exact Haversine check — a massive improvement over scanning all 10 million.

Step 7: Building a Grid Index

Your bounding-box code (Step 5) answers: "What geographic rectangle contains everything within approximately this radius?" A grid index answers: "How can I quickly find only the locations in that rectangle, without scanning the entire database?"

Without an index, given 1,000,000 locations and a bounding box:

```

SELECT * FROM location
WHERE latitude BETWEEN :minLat AND :maxLat
  AND longitude BETWEEN :minLon AND :maxLon;
  
```

Without a useful database index, this may still need to examine a huge number of rows — the bounding box tells you what you're looking for, not how to find it fast.

A grid index changes the storage structure itself. Suppose **CELL_SIZE = 0.01°** (roughly 1 km). Every location gets a cell key:

```
lat = 17.385          int gridX = (int) (latitude * 100);
lon = 78.487          int gridY = (int) (longitude * 100);
                      String key = gridX + ":" + gridY;
                      // e.g. "1738:7848"
x = floor(17.385 / 0.01)
y = floor(78.487 / 0.01)

key = "1738:7848"
```

and a hash map buckets locations by that key:

<pre>grid ├── "1738:7848" → [Location1, Location2, Location3] ├── "1738:7849" → [Location4, Location5] ├── "1739:7848" → [Location6] └── ...</pre>	<pre>Map<String, List<Location>> gridIndex;</pre>
--	---

Insert:

```
gridIndex
.computelfAbsent(key, k -> new ArrayList<>())
.add(location);
```

Query — instead of scanning everything, find the user's cell and check its 8 neighbors (9 cells total):

```
User cell: 1738:7848
Search:    1737:7847 → 1739:7849  (only ~9 cells)
```

Complexity: **O(k)** instead of O(n) — 10 million locations become just a handful of neighboring cells to check

Full GridIndex implementation

```
public class GridIndex {
    private final double CELL_SIZE = 0.01; // ~1km
    private final Map<String, List<Location>> grid = new HashMap<>();

    private String getKey(double lat, double lon) {
        int x = (int) (lat / CELL_SIZE);
        int y = (int) (lon / CELL_SIZE);
        return x + ":" + y;
    }

    public void add(Location loc) {
        String key = getKey(loc.getLatitude(), loc.getLongitude());
        grid.computeIfAbsent(key, k -> new ArrayList<>())
            .add(loc);
    }

    public List<Location> getNearby(double lat, double lon) {
        int x = (int) (lat / CELL_SIZE);
        int y = (int) (lon / CELL_SIZE);

        List<Location> result = new ArrayList<>();

        // check 9 surrounding cells
        for (int i = -1; i <= 1; i++) {
            for (int j = -1; j <= 1; j++) {
                String key = (x + i) + ":" + (y + j);
                if (grid.containsKey(key)) {
                    result.addAll(grid.get(key));
                }
            }
        }
        return result;
    }
}
```

getNearby() **doesn't actually guarantee a radius.** It means "locations contained in these 9 grid cells," not "locations within 1 km" — a location can sit inside one of those 9 cells while still being farther away than the intended radius:

Step 8: A Complete Nearby-Search Service

```
public class LocationService {
    private final GridIndex gridIndex = new GridIndex();

    public void addLocation(Location loc) {
        gridIndex.add(loc);
    }

    public List<Location> findNearby(double lat, double lon, double radiusKm) {
        double[] box = BoundingBoxUtil.getBox(lat, lon, radiusKm);
        double minLat = box[0];
        double maxLat = box[1];
        double minLon = box[2];
        double maxLon = box[3];

        List<Location> candidates = gridIndex.getNearby(lat, lon);

        List<Location> result = new ArrayList<>();
        for (Location loc : candidates) {
            if (loc.getLatitude() < minLat || loc.getLatitude() > maxLat) continue;
            if (loc.getLongitude() < minLon || loc.getLongitude() > maxLon) continue;

            double dist = GeoDistanceUtil.distance(lat, lon, loc.getLatitude(), loc.getLongitude());

            if (dist <= radiusKm) {
                result.add(loc);
            }
        }
        return result;
    }
}
```

Phase 4: GeoHash Indexing

Step 9: Why GeoHash

The grid index from Phase 3 works, but has real limitations at production scale:

- fixed-size cells
- uneven density handling
- poor distribution at scale

GeoHash solves these by giving you:

- ✓ hierarchical indexing (precision is just string length)
- ✓ prefix-based search
- ✓ distributed sharding (shard by hash prefix)
- ✓ fast range queries

Step 10: How GeoHash Encoding Works

GeoHash converts coordinates into strings, where nearby places share a common prefix:

(17.38, 78.48) → "tepgq"

(17.39, 78.49) → "tepgw"

tepgq

tepgw

tepgx ← all nearby, all share the "tepg" prefix

The encoding process:

1. **Start with the full range.** lat: [-90, +90], lon: [-180, +180].
2. **Encode bits.** Repeatedly split the latitude range and the longitude range in half, alternating between them, appending a 1 if the coordinate is in the upper half or a 0 if it's in the lower half, then interleave the two bit sequences.
3. **Base32-encode.** Convert the interleaved binary string into a Base32 string, 5 bits at a time.

Simplified example for lat=17.38, lon=78.48:

(17.38, 78.48) → "tepgq"

(17.39, 78.49) → "tepgw"

tepgq

tepgw

tepgx ← all nearby, all share the "tepg" prefix

Map<String, List<Location>>
geoHashIndex;

Why this is powerful: you can query using simple prefix matching —

LIKE 'tepg%'

Step 11: Implementing GeoHash

Algorithm:

1. Split the latitude range
2. Split the longitude range
3. Encode the resulting bits
4. Convert to Base32

```
public class GeoHash {
    private static final int PRECISION = 12;
    private static final char[] BASE32 = "0123456789bcdefghjkmnpqrstuvwxyz".toCharArray();

    public static String encode(double lat, double lon) {
        double[] latRange = {-90.0, 90.0};
        double[] lonRange = {-180.0, 180.0};

        StringBuilder binary = new StringBuilder();
        boolean isEven = true;

        for (int i = 0; i < PRECISION * 5; i++) {
            if (isEven) {
                double mid = (lonRange[0] + lonRange[1]) / 2;
                if (lon > mid) {
                    binary.append("1");
                    lonRange[0] = mid;
                } else {
                    binary.append("0");
                    lonRange[1] = mid;
                }
            } else {
                double mid = (latRange[0] + latRange[1]) / 2;
                if (lat > mid) {
                    binary.append("1");
                    latRange[0] = mid;
                } else {
                    binary.append("0");
                    latRange[1] = mid;
                }
            }
            isEven = !isEven;
        }
        return toBase32(binary.toString());
    }

    private static String toBase32(String binary) {
        StringBuilder hash = new StringBuilder();
        for (int i = 0; i < binary.length(); i += 5) {
            String chunk = binary.substring(i, i + 5);
            int idx = Integer.parseInt(chunk, 2);
            hash.append(BASE32[idx]);
        }
        return hash.toString();
    }
}
```

Storing locations by hash:

```
public class GeoHashIndex {
    private Map<String, List<Location>> index = new HashMap<>();

    public void add(Location loc) {
        String hash = GeoHash.encode(loc.getLatitude(), loc.getLongitude());
        index.computeIfAbsent(hash, k -> new ArrayList<>())
            .add(loc);
    }
}
```

Step 12: GeoHash-Based Search

A GeoHash search checks the exact cell plus neighboring prefixes

user → geohash prefix → fetch nearby buckets

lat=17.38 lon=78.48 → tepgq

search: tepgq*, tepgp*, tepgw*

```
public List<Location> searchNearby(String hashPrefix) {
    List<Location> result = new ArrayList<>();
    for (String key : index.keySet()) {
        if (key.startsWith(hashPrefix)) {
            result.addAll(index.get(key));
        }
    }
    return result;
}
```

Real use-case flow: a user searches "restaurants near me" → lat/lon → GeoHash → prefix lookup → candidate list, which is then refined using KD-Tree or Haversine — exactly what Phase 5 does next.

Phase 5: KD-Tree for Nearest-Neighbor Search

Step 13: Why KD-Tree

GeoHash (Phase 4) gives you a candidate list efficiently. A KD-Tree goes further and gives you:

- ✓ the exact nearest point
- ✓ fast $O(\log n)$ search
- ✓ dynamic spatial partitioning

Instead of scanning every candidate:

```
(17.3)
/      \
Smaller  bigger

class KNode {
    Location location;
    KNode left;
    KNode right;
}
```

```
if (depth % 2 == 0)
    compare latitude
else
    compare longitude
```

Depth	Axis
0	latitude
1	longitude
2	latitude

Complexity: $O(\log n)$ on average.

Step 14: Building a KD-Tree

Build algorithm:

```
public class KDTree {
    private KNode root;
    public KNode build(List<Location> points, int depth) {
        if (points.isEmpty()) return null;
        int axis = depth % 2;

        points.sort((a, b) -> axis == 0
            ? Double.compare(a.getLatitude(), b.getLatitude())
            : Double.compare(a.getLongitude(), b.getLongitude()));

        int median = points.size() / 2;

        KNode node = new KNode();
        node.location = points.get(median);

        node.left = build(points.subList(0, median), depth + 1);
        node.right = build(points.subList(median + 1, points.size()),
            depth + 1);

        return node;
    }
}
```

Step 15: Nearest-Neighbor Search

Algorithm:

1. Traverse the likely branch first (go left or right depending on the query point).
2. Track the nearest distance found so far.
3. Backtrack and check the other branch if it could still contain a closer point.

Worked example — query 17.5, 78.4 against the tree built above: go to **A**, check **B** and **C**, update the running best. Complexity: $O(\log n)$.

```
public class KDTreeSearch {
    private Location best;
    private double bestDist = Double.MAX_VALUE;

    public Location nearest(KDNode node, lat, lon, int depth) {
        if (node == null) return best;
        double dist = distance(lat, lon, node.location.getLatitude(), node.location.getLongitude());
        if (dist < bestDist) {
            bestDist = dist;
            best = node.location;
        }

        int axis = depth % 2;
        KDNode next, other;

        if (axis == 0) {
            if (lat < node.location.getLatitude()) { next = node.left; other = node.right; }
            else { next = node.right; other = node.left; }
        } else {
            if (lon < node.location.getLongitude()) { next = node.left; other = node.right; }
            else { next = node.right; other = node.left; }
        }

        nearest(next, lat, lon, depth + 1);

        // check if we must explore the other side too
        if (shouldCheckOther(node, lat, lon, axis)) { nearest(other, lat, lon, depth + 1); }

        return best;
    }

    private boolean shouldCheckOther(KDNode node, lat, lon, int axis) {
        return true; // simplified for concept
    }
}
```

Step 16: Combining GeoHash and KD-Tree

This is the real production flow, combining everything so far:

```
Step 1 – User request:           "find nearest restaurants"  
Step 2 – GeoHash filter:        10,000,000 → 5,000 candidates  
Step 3 – KD-Tree search:        5,000 → top 10 nearest  
Step 4 – Haversine final check: exact ranking of those 10
```

Final result:

```
1. Biryani Hub   (1.2 km)  
2. Spice Villa  (1.6 km)  
3. Food Street  (2.0 km)
```

Phase 6: Persistence

Everything so far has lived only in memory. This phase persists it to disk.

Step 17: File Storage

Store as JSON:

```
[
  {
    "id": 1,
    "lat": 17.3,
    "lon": 78.4
  }
]
```

Step 18: Custom Binary Format

Store directly as bytes for much faster reads:

```
[id][lat][lon]
```

```
DataOutputStream out = new DataOutputStream(file);
```

```
out.writeLong(id);
out.writeDouble(lat);
out.writeDouble(lon);
```

Why: faster disk reads, more compact storage, and more cache-friendly than parsing JSON on every load.

Phase 7: Persistence

A real service needs to support many simultaneous reads and writes safely. Reach for:

- **ReadWriteLock** — cheap reads, exclusive writes
- **ConcurrentHashMap** — for the index structures themselves (grid buckets, GeoHash buckets)

Phase 8: Production-Grade Features

Step 19: Polygon Search

Check whether a point lies inside an arbitrary polygon — "find users inside a city boundary," "restaurants inside a delivery zone."

Algorithm: **Ray casting**, or the winding number method.

Step 20: Routing

Represent the road network as a graph: Each road (edge) has a weight — distance, time, or live traffic. Real routing engines run:

- **Dijkstra** — shortest path, guaranteed optimal
- **A*** — heuristic-guided, usually faster in practice than Dijkstra for a single query

Phase 9: Rendering GeoHash Boxes & Grids on Map

Every GeoHash, grid cell is just numbers — a hash string, a pair of (lat, lng) ranges, a median coordinate. None of that is visible on a map by itself. "Drawing" it means one specific, repeatable conversion:

```
index      → a pair of (lat,lng) corners      → a Leaflet layer object → pixels on screen
("tepgq6") (minLat/maxLat/minLng/maxLng)    L.rectangle(...)      a green box you
```

Step 9.1: Decoding a Hash Back Into a Box

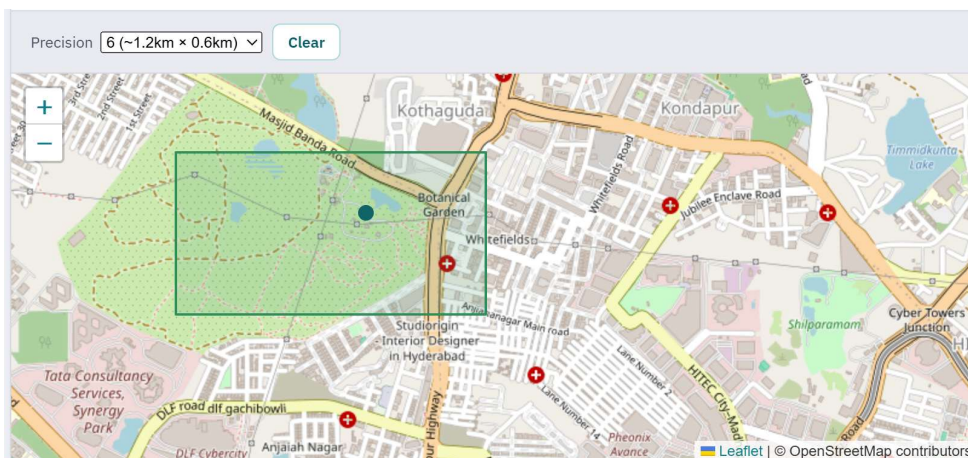
Decoding just runs the same bisection forward again, except instead of asking "which half is the point in," it reads the answer back out of the hash's bits and shrinks the range accordingly. This is `geohashBounds()` from the simulator, unmodified:

```
// geo-delivery-simulator/index.html — geohashBounds()
function geohashBounds(hash) {
  let latRange = [-90, 90], lngRange = [-180, 180];
  let evenBit = true;
  for (const c of hash) {
    const idx = GEOHASH_BASE32.indexOf(c); // 1 character = 5 bits
    for (let n = 4; n >= 0; n--) {
      const bitN = (idx >> n) & 1; // read one bit at a time, high to low
      if (evenBit) { // even bit position → this bit was a longitude split
        const mid = (lngRange[0] + lngRange[1]) / 2;
        if (bitN === 1) { lngRange[0] = mid; } else { lngRange[1] = mid; }
      } else { // odd bit position → this bit was a latitude split
        const mid = (latRange[0] + latRange[1]) / 2;
        if (bitN === 1) { latRange[0] = mid; } else { latRange[1] = mid; }
      }
      evenBit = !evenBit;
    }
  }
  return { minLat: latRange[0], maxLat: latRange[1], minLng: lngRange[0], maxLng: lngRange[1] };
}
```

Walk it once by hand for a 2-character hash, precision-shortened for clarity: start with the whole planet, $\text{lat} \in [-90, 90]$, $\text{lng} \in [-180, 180]$. Each of the 10 bits ($2 \text{ chars} \times 5 \text{ bits}$) halves either the latitude or the longitude range, alternating, starting with longitude:

bit 1 (lng) → lng shrinks to $[0, 180]$ or $[-180, 0]$
bit 2 (lat) → lat shrinks to $[0, 90]$ or $[-90, 0]$
bit 3 (lng) → lng range halves again within whichever half bit 1 picked
bit 4 (lat) → lat range halves again within whichever half bit 2 picked
... (10 bits total for 2 characters)
result: a small lat/lng rectangle — NOT a point, a box

Precision is nothing but "how many times did we bisect" — every extra character is 5 more bisections, which is why cell size roughly quarters (halves in both dimensions) every ~2 characters.



clicked: (17.455234, 78.361230)

geohashEncode(...) → teper1

geohashBounds("teper1") →

lat: [17.451782, 17.457275]

lng: [78.354492, 78.365479]

Step 9.2: The Coordinate-Order Trap

Once you have $\{\text{minLat}, \text{maxLat}, \text{minLng}, \text{maxLng}\}$, drawing it is one line:

```
const b = geohashBounds(hash);
L.rectangle([[b.minLat, b.minLng], [b.maxLat, b.maxLng]], {
  color: '#2f8f5b', weight: 1, fillOpacity: 0.03, dashArray: '3,3'
}).addTo(map);
```

Step 9.3: Drawing the Whole Grid

Sampling the Cell Size Instead of Computing It

A grid needs to know the cell width and height in degrees before it can start stepping across the viewport. You could derive that arithmetically from the bit-allocation rule (how many of the `precision × 5` bits went to longitude vs. latitude), but the actual code takes a much simpler shortcut: encode one point, decode it right back, and measure the box you get. Every cell at a given precision is the same size, so one sample tells you the step size for the entire grid:

```
// geo-delivery-simulator/index.html — drawGeohashGrid(), first few lines
const sample = geohashBounds(geohashEncode((sw.lat + ne.lat) / 2, (sw.lng + ne.lng) / 2,
precision));
const latStep = sample.maxLat - sample.minLat;
const lngStep = sample.maxLng - sample.minLng;
```

Step 9.4: Walking the Viewport in Cell-Sized Steps

With a known step size, covering the visible map is a plain nested loop over the current `map.getBounds()`, re-encoding at each sample point, skipping any hash already seen (adjacent samples inside the same cell re-encode to the identical string), and drawing whatever's new:

```
const seen = new Set();
for (let lat = sw.lat; lat <= ne.lat + latStep; lat += latStep) {
  for (let lng = sw.lng; lng <= ne.lng + lngStep; lng += lngStep) {
    const hash = geohashEncode(lat, lng, precision);
    if (seen.has(hash)) continue;
    seen.add(hash);
    const b = geohashBounds(hash);
    const rect = L.rectangle([[b.minLat, b.minLng], [b.maxLat, b.maxLng]], {
      color: '#2f8f5b', weight: 1, fillOpacity: 0.03, dashArray: '3,3'
    });
    rect.bindTooltip(hash, { permanent: true, direction: 'center' });
    gridLayer.addLayer(rect);
  }
}
```

Note what this loop is not doing: it never computes a cell's corners by hand. It samples a point, asks GeoHash where that point belongs, and lets `geohashBounds` answer with the exact rectangle — the same "encode, then decode" round trip from Phase 1, just run in a loop instead of once per click.

Step 9.5: Redrawing on Pan, Zoom, and Precision Change

A grid computed once goes stale the moment the user drags the map — so the whole draw routine re-runs on Leaflet's own `moveend` event (fires after a pan or zoom settles), and again whenever the precision dropdown changes:

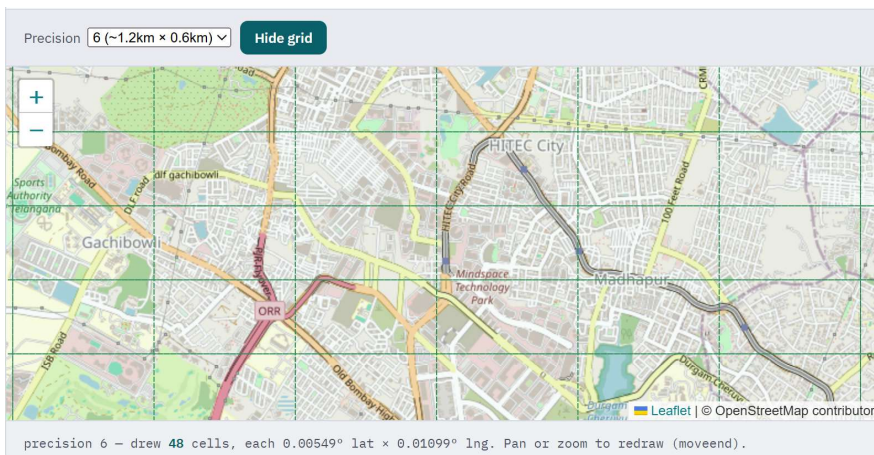
```
document.getElementById('geohashPrecision').addEventListener('change', () => {
    if (gridVisible) drawGeohashGrid();
});
map.on('moveend', () => {
    if (gridVisible) drawGeohashGrid();
});
```

Each call clears the previous grid layer's contents first (`gridLayer.clearLayers()`) rather than accumulating rectangles — otherwise every pan would leave the old grid behind, stacked under the new one.

Step 9.6: The Cell-Count Guard Rail

Zoom out far enough at high precision (say, precision 8 showing the whole city) and the nested loop above would try to draw tens of thousands of tiny rectangles — slow to compute, and unreadable even if it rendered instantly. The code estimates the cell count before looping and bails with a message instead:

```
const maxCells = 500;
const estCells = Math.ceil((ne.lat - sw.lat) / latStep) * Math.ceil((ne.lng - sw.lng) / lngStep);
if (estCells > maxCells) {
    L.popup().setLatLng(map.getCenter()).setContent('Zoom in for a readable GeoHash grid at this precision.').openOn(map);
    return;
}
```



This is the same "zoom-in-first" behavior every slippy map with a grid overlay uses (vector tile styles do the same thing with label density) — it's not GeoHash-specific, but it's the detail that keeps a naive "draw every cell in view" approach from ever becoming a real performance problem.

Step 9.7: Precision & Cell Size

Every extra character adds 5 bits, alternating longitude/latitude, so cell dimensions roughly halve every character in one axis and every other character in the other:

Precision (chars)	Bits	Cell width × height (approx, at the equator)
4	20	~39.1 km × 19.5 km
5	25	~4.9 km × 4.9 km
6	30	~1.2 km × 0.61 km
7	35	~153 m × 153 m
8	40	~38.2 m × 19 m
9	45	~4.8 m × 4.8 m

Two things worth internalizing from this table, both visible in the demos above:

- **Cells are never square in meters except at the equator** — longitude bits and latitude bits are interleaved evenly, but a degree of longitude is narrower than a degree of latitude everywhere except the equator, so odd precisions in particular (5, 7, 9...) produce visibly rectangular, not square, cells.
- **The cell size is fixed once precision is fixed** — unlike the bounding-box radius math in LOCATION_DB.html's Phase 2 (which depends on $\cos(\text{lat})$ and the requested radius), a GeoHash cell at a given precision is exactly the same size everywhere on Earth at that latitude band. That uniformity is the entire reason GeoHash exists as an upgrade over a naive lat/long grid (LOCATION_DB.html Phase 3's closing note) — and it's also the thing that makes a real grid legible enough to draw at all.

Phase 10: Drawing a KD-Tree's Spatial Partition

A KD-Tree build alternates the split axis by depth (latitude at depth 0, longitude at depth 1, latitude again at depth 2, ...) and splits at the median point along that axis. Spatially, each split is a straight line through the map: a latitude-axis split is a horizontal line at the median latitude (bounded to the current node's slice of the map); a longitude-axis split is a vertical line at the median longitude.

Recursive Splitting Lines:

This is the same recursive shape as the Java build() method in LOCATION_DB.html, except each call also draws a line instead of only linking nodes:

```
// bbox = { minLat, maxLat, minLng, maxLng } — the slice of the map this node owns
function buildAndDraw(points, depth, bbox, layerGroup) {
  if (points.length === 0) return;
  const axis = depth % 2; // 0 = split on latitude, 1 = split on longitude

  points.sort((a, b) => axis === 0 ? a.lat - b.lat : a.lng - b.lng);
  const medianIdx = Math.floor(points.length / 2);
  const median = points[medianIdx];

  if (axis === 0) {
    // horizontal line at median.lat, spanning this node's longitude slice
    L.polyline([[median.lat, bbox.minLng], [median.lat, bbox.maxLng]],
      { color: splitColor(depth), weight: 2, opacity: 0.8 }).addTo(layerGroup);
  } else {
    // vertical line at median.lng, spanning this node's latitude slice
    L.polyline([[bbox.minLat, median.lng], [bbox.maxLat, median.lng]],
      { color: splitColor(depth), weight: 2, opacity: 0.8 }).addTo(layerGroup);
  }

  const left = points.slice(0, medianIdx);
  const right = points.slice(medianIdx + 1);

  if (axis === 0) {
    buildAndDraw(left, depth + 1, { ...bbox, maxLat: median.lat }, layerGroup); // below the line
    buildAndDraw(right, depth + 1, { ...bbox, minLat: median.lat }, layerGroup); // above the line
  } else {
    buildAndDraw(left, depth + 1, { ...bbox, maxLng: median.lng }, layerGroup); // left of the line
    buildAndDraw(right, depth + 1, { ...bbox, minLng: median.lng }, layerGroup); // right of the line
  }
}
```

The key idea that makes this different from the GeoHash grid above: a KD-Tree's regions are not uniform rectangles. Every split's line length is clipped to the bounding box its parent split left behind, so dense clusters of points get many short,